

**LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 3: BLADE TEMPLATE LARAVEL**



**Universitas
Telkom**

Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

**UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026**

1. Tujuan

Adapun tujuan yang ingin dicapai pada praktikum Modul 3 ini adalah sebagai berikut:

1. Memahami konsep Blade Template sebagai template engine bawaan Laravel, termasuk cara kerja kompilasi file `.blade.php` menjadi kode PHP yang disimpan pada cache.
2. Mampu memakai sintaks dasar Blade, yaitu `{{ }}` untuk menampilkan data serta directive percabangan (`@if/@else`) dan perulangan (`@foreach/@forelse`) untuk mengontrol tampilan.
3. Mampu menyusun layout utama dengan `@extends`, `@section`, dan `@yield` sehingga bagian halaman yang sama cukup ditulis satu kali.
4. Mampu membuat komponen Blade reusable (`<x-...>`) yang dipanggil lewat atribut, serta memakai partial (`@include`) untuk memecah view yang panjang.
5. Mampu menghubungkan route, controller, dan view Blade dalam pola MVC lalu menguji hasilnya melalui browser.
6. Mampu menganalisis alasan pembagian layout, komponen, dan partial beserta kendala yang muncul selama pengerjaan.

2. Alat dan Bahan

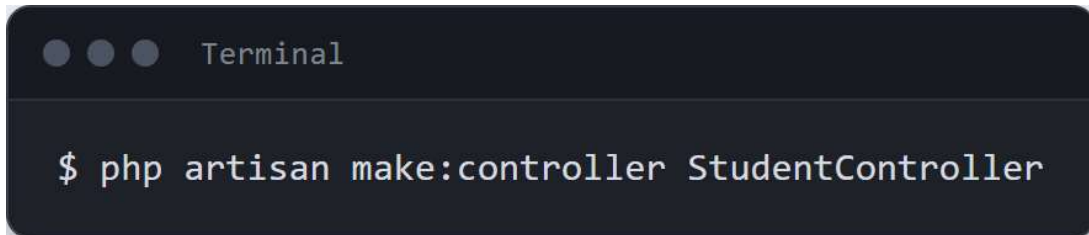
Praktikum ini dikerjakan menggunakan satu buah laptop dengan prosesor Intel Core i7-13650HX dan sistem operasi Windows 11. Perangkat lunak yang digunakan meliputi Laragon sebagai lingkungan pengembangan web lokal, PHP versi 8.3.30 sebagai bahasa pemrograman sisi server, Composer versi 2.10.3 sebagai dependency manager, Laravel Framework versi 13.33.0 sebagai framework utama aplikasi, Visual Studio Code sebagai teks editor untuk menuliskan kode program, Git sebagai version control untuk mengelola kode sumber, serta Google Chrome sebagai web browser untuk menjalankan dan menguji aplikasi.

3. Langkah-Langkah Praktikum

Langkah-langkah praktikum ini diuraikan sesuai materi praktikum 3.4.2 sampai 3.4.4, yaitu (1) membangun layout utama beserta controller dan route untuk menampilkan daftar mahasiswa, (2) membuat komponen Blade reusable berupa kotak alert, dan (3) memecah tampilan kartu mahasiswa menjadi partial yang dipanggil dengan `@include`.

3.1 Membuat Controller

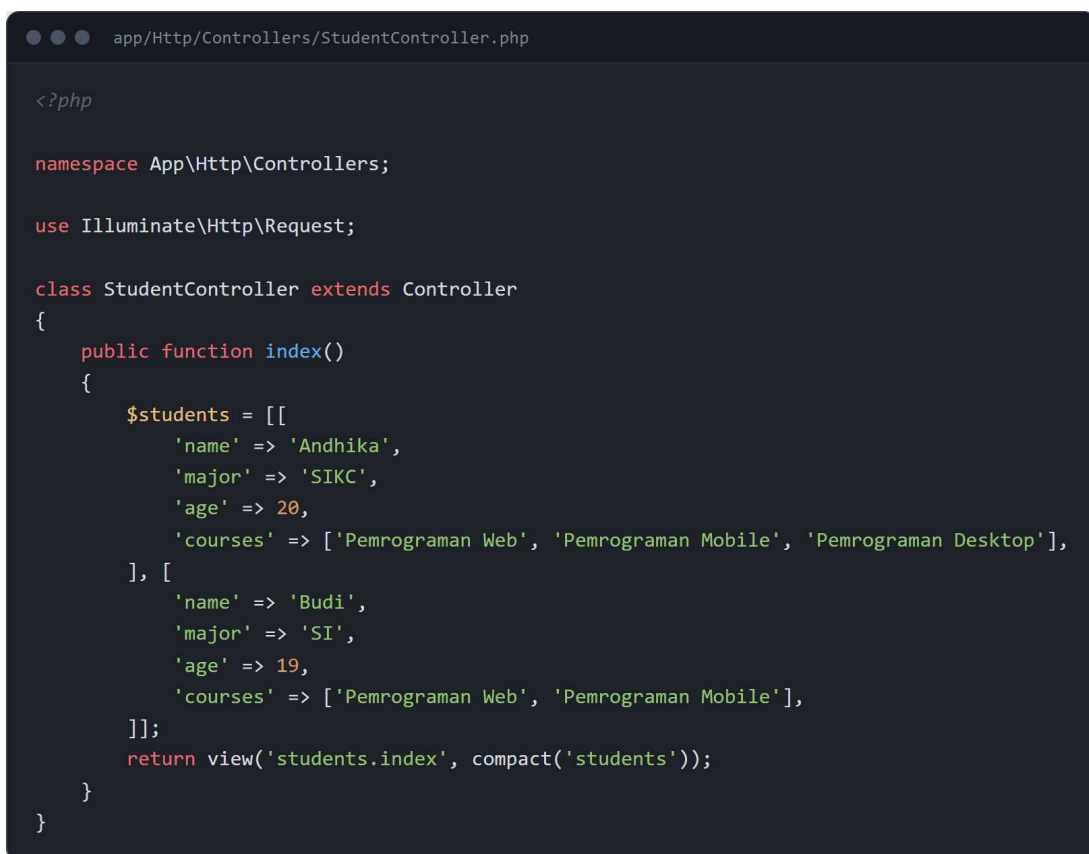
Controller dibuat melalui perintah artisan berikut, yang secara otomatis menghasilkan file `app/Http/Controllers/StudentController.php` beserta kerangka class-nya:



```
Terminal

$ php artisan make:controller StudentController
```

Isi file `app/Http/Controllers/StudentController.php` adalah sebagai berikut:



```
app/Http/Controllers/StudentController.php

<?php

namespace App\Http\Controllers;

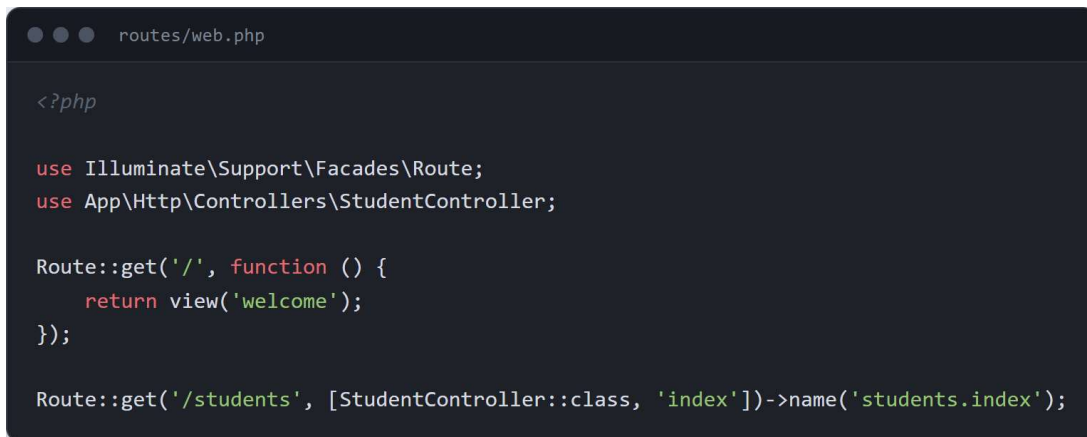
use Illuminate\Http\Request;

class StudentController extends Controller
{
    public function index()
    {
        $students = [[
            'name' => 'Andhika',
            'major' => 'SIKC',
            'age' => 20,
            'courses' => ['Pemrograman Web', 'Pemrograman Mobile', 'Pemrograman Desktop'],
        ], [
            'name' => 'Budi',
            'major' => 'SI',
            'age' => 19,
            'courses' => ['Pemrograman Web', 'Pemrograman Mobile'],
        ]];
        return view('students.index', compact('students'));
    }
}
```

Method `index()` menyiapkan array multidimensi berisi dua data mahasiswa, yaitu nama, jurusan, umur, dan daftar mata kuliah, lalu mengirimkannya ke view `students.index` dengan helper `compact('students')`. Karena `compact('students')` setara dengan `['students' => $students]`, variabel `$students` dapat langsung dipanggil di dalam Blade. Data masih berupa array biasa karena praktikum ini belum menggunakan database, tetapi cara mengirim data ke view tetap sama.

3.2 Mendefinisikan Route

Route didefinisikan pada file routes/web.php seperti berikut:



```
routes/web.php

<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/students', [StudentController::class, 'index'])->name('students.index');
```

use App\Http\Controllers\StudentController meng-import controller agar dapat dipakai, Route::get('/students', ...) membuat route GET /students yang menjalankan method index() pada StudentController, dan ->name('students.index') memberi nama route sehingga dapat dipanggil di Blade dengan route('students.index'). Berbeda dengan Modul 2 yang banyak memakai closure, pada praktikum ini route diarahkan ke method controller agar logika aplikasi terpisah dari file routes/web.php.

3.3 Membuat Layout Utama

Layout utama dibuat pada file resources/views/layouts/main.blade.php. File ini berfungsi sebagai cetakan halaman: bagian yang selalu sama (header, navigasi, dan footer) ditulis sekali, sedangkan bagian yang berbeda ditandai dengan @yield:

```
resources/views/layouts/main.blade.php

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title')</title>
</head>

<body>
    <header>
        <h1>Praktikum Laravel</h1>
        <nav>
            <a href="{{ route('students.index') }}">Mahasiswa</a>
        </nav>
    </header>

    <main>
        @yield('content')
    </main>

    <footer>
        <p>&copy; 2025 Praktikum Laravel - Andhika Yuridho</p>
    </footer>
</body>

</html>
```

@yield('title') dan @yield('content') adalah placeholder judul dan konten yang akan diisi oleh view turunan. Link navigasi memakai named route route('students.index'), sehingga jika alamat URL berubah cukup mengubah definisi route saja. Dengan layout ini, perubahan menu atau footer hanya perlu dilakukan pada satu file.

3.4 Membuat View Daftar Mahasiswa

View daftar mahasiswa dibuat pada file resources/views/students/index.blade.php. View ini mewarisi layout utama lalu mengisi kedua placeholder tersebut dengan data dari controller:

```
resources/views/students/index.blade.php (tahap 3.4.2)

@extends('layouts.main')
@section('title', 'Daftar Mahasiswa')
@section('content')
<h2>Daftar Mahasiswa</h2>
    @foreach($students as $student)
    <div class="card">

        <h3>{{ $student['name'] }}</h3>
        <p><strong>Jurusan: </strong>{{ $student['major'] }}</p>
        <p><strong>Umur: </strong>{{ $student['age'] }}</p>

        @if($student['age'] >= 22)
        <p>Status: <span style="color: green;">Senior</span></p>
        @else
        <p>Status: <span style="color: blue;">Junior</span></p>
        @endif

        <h4>Mata Kuliah: </h4>
        @forelse($student['courses'] as $course)
        <span class="course">{{ $course }}</span>
        @empty
        <p>Tidak ada mata kuliah terdaftar.</p>
        @endforelse
    </div>
    @endforeach @endsection
```

@extends('layouts.main') memanggil layout utama, @section('title', 'Daftar Mahasiswa') mengisi placeholder judul, dan blok @section('content') sampai @endsection mengisi konten halaman. Di dalam konten, directive @foreach melakukan perulangan pada data \$students, sintaks {{ \$student['name'] }} menampilkan nama mahasiswa, @if menentukan status Senior jika umur minimal 22 tahun dan Junior untuk selain itu, serta @forelse menampilkan daftar mata kuliah dengan pesan cadangan @empty jika kosong. Setiap directive wajib ditutup dengan pasangannya, misalnya @if dengan @endif dan @foreach dengan @endforeach.

3.5 Membuat Komponen Blade (Reusable Components)

Komponen Blade dibuat pada folder `resources/views/components` dan dipanggil dengan tag berawalan `x-`. File `alert.blade.php` berisi kotak pesan yang warnanya ditentukan lewat atribut `type`:

```
resources/views/components/alert.blade.php

@if ($type === 'error')
<div style="padding: 10px; border: 1px solid red; margin: 10px 0;">
  @else
    <div style="padding: 10px; border: 1px solid green; margin: 10px 0;">
      @endif
      <strong>{{ ucfirst($type) }}:</strong> {{ $message }}
    </div>
  </div>
</div>
```

Operator ternary pada `{{ $type === 'error' ? 'red' : 'green' }}` menentukan warna border merah atau hijau, sedangkan `ucfirst($type)` menampilkan huruf pertama menjadi kapital (contoh: `success` menjadi `Success`). File di folder `components` otomatis dikenali sebagai komponen `<x-alert>`. Komponen dipanggil dari view mahasiswa seperti berikut:

```
resources/views/students/index.blade.php (pemanggilan komponen)

<x-alert type="success" message="Data mahasiswa berhasil dimuat." />
```

Nilai atribut `type` dan `message` diambil oleh komponen sebagai variabel `$type` dan `$message`. Satu komponen cukup dibuat sekali, lalu dapat dipanggil berkali-kali dengan atribut berbeda. Inilah yang dimaksud `reusable`: jika desain alert ingin diubah, cukup ubah file `alert.blade.php` dan seluruh halaman yang memakainya akan ikut berubah.

3.6 Membuat Partial Sederhana

Partial adalah potongan file Blade yang disisipkan ke view lain menggunakan `@include`. Pada bagian ini, tampilan satu kartu mahasiswa dipisahkan ke file `resources/views/partials/student-card.blade.php`:

```
resources/views/partials/student-card.blade.php

<div class="card">
    <h3>{{ $student['name'] }}</h3>
    <p><strong>Jurusan: </strong>{{ $student['major'] }}</p>
    <p><strong>Umur: </strong>{{ $student['age'] }} tahun</p>

    @if($student['age'] >= 22)
    <p>Status: <span style="color: green;">Senior</span></p>
    @else
    <p>Status: <span style="color: blue;">Junior</span></p>
    @endif

    <h4>Mata Kuliah: </h4>
    @forelse($student['courses'] as $course)
    <span class="course">{{ $course }}</span>
    @empty
    <p>Tidak ada mata kuliah terdaftar.</p>
    @endforelse
</div>
```

Partial ini hanya berfokus pada tampilan satu mahasiswa. Variabel \$student yang dipakai di dalamnya tidak didefinisikan di sana, melainkan dikirim dari view pemanggil. View utama resources/views/students/index.blade.php kemudian diubah menjadi seperti berikut:

```

resources/views/students/index.blade.php (setelah partial)

@extends('layouts.main')
@section('title', 'Daftar Mahasiswa')
@section('content')
<h2>Daftar Mahasiswa</h2>

<x-alert type="success" message="Data mahasiswa berhasil dimuat." />

<!-- @foreach($students as $student)
<div class="card">

    <h3>{{ $student['name'] }}</h3>
    <p><strong>Jurusan: </strong>{{ $student['major'] }}</p>
    <p><strong>Umur: </strong>{{ $student['age'] }}</p>

    @if($student['age'] >= 22)
    <p>Status: <span style="color: green;">Senior</span></p>
    @else
    <p>Status: <span style="color: blue;">Junior</span></p>
    @endif

    <h4>Mata Kuliah: </h4>
    @forelse($student['courses'] as $course)
    <span class="course">{{ $course }}</span>
    @empty
    <p>Tidak ada mata kuliah terdaftar.</p>
    @endforelse
</div>
@endforeach -->

@foreach($students as $student)
@include('partials.student-card', ['student' => $student])
@endforeach
@endsection

```

`@include('partials.student-card', ['student' => $student])` memanggil partial `student-card` dan mengoper variabel `$student` ke dalamnya, sehingga blok kartu yang sebelumnya ditulis langsung di view utama tidak perlu diulang. Bedanya dengan `<x-alert>`, `@include` tidak memiliki konsep props/atribut seperti komponen, melainkan langsung mengirim variabel. Dengan cara ini kode jadi lebih bersih karena view utama tidak terlalu panjang.

3.7 Menjalankan dan Menguji Aplikasi

Aplikasi dijalankan melalui development server Laravel dengan perintah `php artisan serve`, lalu alamat `http://127.0.0.1:8000/students` dibuka pada browser. Route `/students` diproses oleh `StudentController`, view mahasiswa diwarisi dari layout utama, komponen alert dirender, dan tiap data mahasiswa ditampilkan melalui partial. Halaman berhasil merender kedua data mahasiswa tanpa galat, dan hasil pengujian ditunjukkan pada bagian berikutnya.

4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar aplikasi yang berjalan pada browser melalui `php artisan serve` pada alamat `http://127.0.0.1:8000/students`.

Praktikum Laravel

[Mahasiswa](#)

Daftar Mahasiswa

Success: Data mahasiswa berhasil dimuat.

Andhika

Jurusan: SIKC

Umur: 20

Status: [Junior](#)

Mata Kuliah:

Pemrograman Web Pemrograman Mobile Pemrograman Desktop

Budi

Jurusan: SI

Umur: 19

Status: [Junior](#)

Mata Kuliah:

Pemrograman Web Pemrograman Mobile

© 2025 Praktikum Laravel - Andhika Yuridho

Gambar 1. Halaman daftar mahasiswa pada alamat `/students`

Gambar 1 menampilkan halaman daftar mahasiswa yang dihasilkan oleh route `/students`. Judul halaman Daftar Mahasiswa berasal dari `@yield('title')` yang diisi view turunan, header berisi Praktikum Laravel dan link navigasi Mahasiswa diwarisi dari `layouts/main.blade.php`, lalu konten diisi dengan daftar dua mahasiswa, yaitu Andhika berjurusan SIKC berumur 20 tahun dan Budi berjurusan SI berumur 19 tahun. Keduanya berstatus Junior karena umurnya belum mencapai 22 tahun, dan footer menampilkan copyright pembuat halaman.

Daftar Mahasiswa

Success: Data mahasiswa berhasil dimuat.

Gambar 2. Komponen `<x-alert>` pada halaman daftar mahasiswa

Gambar 2 memperlihatkan pemanggilan komponen `<x-alert type="success" message="Data mahasiswa berhasil dimuat." />` pada view. Karena atribut `type` berisi `success`, operator ternary di dalam `alert.blade.php` menghasilkan warna border hijau, teks `Success:` tampil kapital berkat `ucfirst($type)`, dan pesan dari atribut `message` dirender di dalam kotak alert. Komponen yang sama dapat dipanggil lagi dengan `type="error"` untuk mendapatkan kotak merah tanpa mengubah file komponen.

Andhika

Jurusan: SIKC

Umur: 20

Status: Junior

Mata Kuliah:

Pemrograman Web Pemrograman Mobile Pemrograman Desktop

Gambar 3. Kartu mahasiswa yang dirender melalui partial

Gambar 3 menampilkan satu kartu mahasiswa hasil `@include('partials.student-card')`. Kartu memuat nama, jurusan, umur, status, serta daftar mata kuliah yang ditampilkan oleh `@forelse`. Tampilannya sama persis dengan sebelum file dipisah menjadi partial, sehingga pemecahan view tidak mengubah hasil render halaman.

5. Analisis dan Pembahasan

Tampilan aplikasi pada praktikum ini dibagi menjadi tiga tingkatan dengan alasan yang berbeda. Layout utama (`main.blade.php`) memegang bagian yang selalu sama di setiap halaman, yaitu header, navigasi, dan footer, sehingga perubahan menu atau gaya halaman cukup dilakukan pada satu file dan otomatis berlaku untuk semua halaman yang mewarisi layout. Komponen (`<x-alert>`) dipilih untuk elemen antarmuka kecil yang dipakai berulang di banyak halaman dan menerima atribut, sehingga perilaku alert dapat bervariasi per pemanggilan tanpa menduplikasi markup. Partial (`student-card`) dipakai untuk memecah bagian halaman yang besar agar view utama tetap ringkas; partial menerima variabel langsung lewat `@include` sehingga cocok untuk satu blok tampilan yang dirender per data.

Perbedaan sebelum dan sesudah menggunakan Blade terlihat pada cara menyusun halaman. Pada Modul 2, setiap halaman dibuat sebagai file Blade mandiri sehingga tag HTML dasar dan

navigasi harus ditulis ulang di setiap file. Setelah memakai layout, bagian tersebut hanya ada di `main.blade.php` dan halaman lain cukup memanggil `@extends` lalu mengisi `@section`. Perubahan menu kini langsung berlaku ke seluruh halaman, sedangkan tampilan akhir di browser tetap sama ketika kartu mahasiswa dipindahkan ke partial, karena pemecahan file hanya mengatur struktur kode bukan hasil render.

Kendala yang ditemui selama pengerjaan antara lain: (1) galat `Undefined array key` pada view karena nama key array di Blade tidak sama dengan key yang didefinisikan di controller (misalnya penulisan `name`, `age`, atau `ages`), yang tercatat pada file `students/index.blade.php` dan `partials/student-card.blade.php`, lalu diselesaikan dengan menyesuaikan penulisan key di view dengan data pada `StudentController`; (2) galat sintaks `unexpected end of file` dan `unexpected token extends` karena directive tidak ditutup dengan benar atau `@extends` tidak berada di posisi paling atas, yang diselesaikan dengan melengkapi `@endforelse`, `@endforeach`, dan `@endif` lalu menempatkan `@extends` sebelum section lain. Setelah perbaikan tersebut, halaman `/students` merender kedua data mahasiswa tanpa galat, sebagaimana dibuktikan pada tangkapan layar bagian sebelumnya.

6. Kesimpulan

Berdasarkan praktikum Modul 3 ini, saya memahami Blade Template sebagai template engine bawaan Laravel yang menyusun tampilan pada file `resources/views` dengan ekstensi `.blade.php`. Sintaks dasar `{{ }}` berhasil digunakan untuk menampilkan data dari controller, dan directive percabangan maupun perulangan (`@if`, `@else`, `@foreach`, `@forelse`) berhasil mengontrol tampilan status mahasiswa serta daftar mata kuliah. Pola layout dengan `@extends`, `@section`, dan `@yield` juga berhasil diterapkan sehingga header, navigasi, dan footer cukup ditulis satu kali pada `layouts/main.blade.php`.

Selain itu, komponen `<x-alert>` berhasil dibuat sebagai elemen reusable yang dipanggil lewat atribut `type` dan `message`, serta partial `student-card` berhasil dipanggil dengan `@include` lengkap dengan pengiriman variabel `$student` sehingga view utama menjadi lebih ringkas. Seluruh halaman telah diakses melalui route `/students` yang mengarah ke method `index()` pada `StudentController`, sehingga alur route, controller, dan view berjalan dalam pola MVC. Melalui praktikum ini terbukti bahwa pembagian layout, komponen, dan partial membuat kode tampilan lebih teratur, tidak duplikatif, dan mudah dirawat.

7. Link Repository Github

<https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/praktikum3>